18. Split BST

Problem Statement

Write a Recursive Approach to Split BST.

Given a Binary Search Tree (BST) and a target value, split the BST into two subtrees where one subtree contains nodes with values less than the target value, and the other subtree contains nodes with values greater than or equal to the target value. Return the two root nodes of the resulting subtrees.

Examples

Example 1:

- Input:
  - Binary Search Tree:

    ```
         4
        / \
       2   6
      / \   \
     1   3    7
    ```

  - Target: 5
- Output:
  - Split BST 1:
    - Tree 1 (Greater):

      ```
        6
         \
          7
      ```

    - Tree 2 (Smaller):

      ```
          4
         / \
        2   5
       /
      1
       \
        3
      ```

- Explanation:
  - In this example, the binary search tree is split based on the target value of 5.
  - All nodes greater than the target value (6, 7) are moved to Tree 1, while all nodes smaller than or equal to the target value (4, 2, 5, 1, 3) are moved to Tree 2.
  - The resulting Tree 1 contains nodes greater than 5, while Tree 2 contains nodes smaller than or equal to 5.

Example 2:

- Input:
  - Binary Search Tree:

    ```
         4
        / \
       2   6
      / \   \
     1   3    7
    ```

  - Target: 3
- Output:
  - Split BST 2:
    - Tree 1 (Greater):

      ```
        4
         \
          6
           \
            7
      ```

    - Tree 2 (Smaller):

      ```
         2
        / \
       1   3
      ```

- Explanation:
  - In this example, the binary search tree is split based on the target value of 3.
  - All nodes greater than the target value (4, 6, 7) are moved to Tree 1, while all nodes smaller than or equal to the target value (2, 1, 3) are moved to Tree 2.
  - The resulting Tree 1 contains nodes greater than 3, while Tree 2 contains nodes smaller than or equal to 3.

Example 3:

- Input:
  - Binary Search Tree:

    ```
         4
        / \
       2   6
      / \   \
     1   3    7
    ```

  - Target: 7
- Output:
  - Split BST 3:
    - Tree 1 (Greater):

      ```
        7
      ```

    - Tree 2 (Smaller):

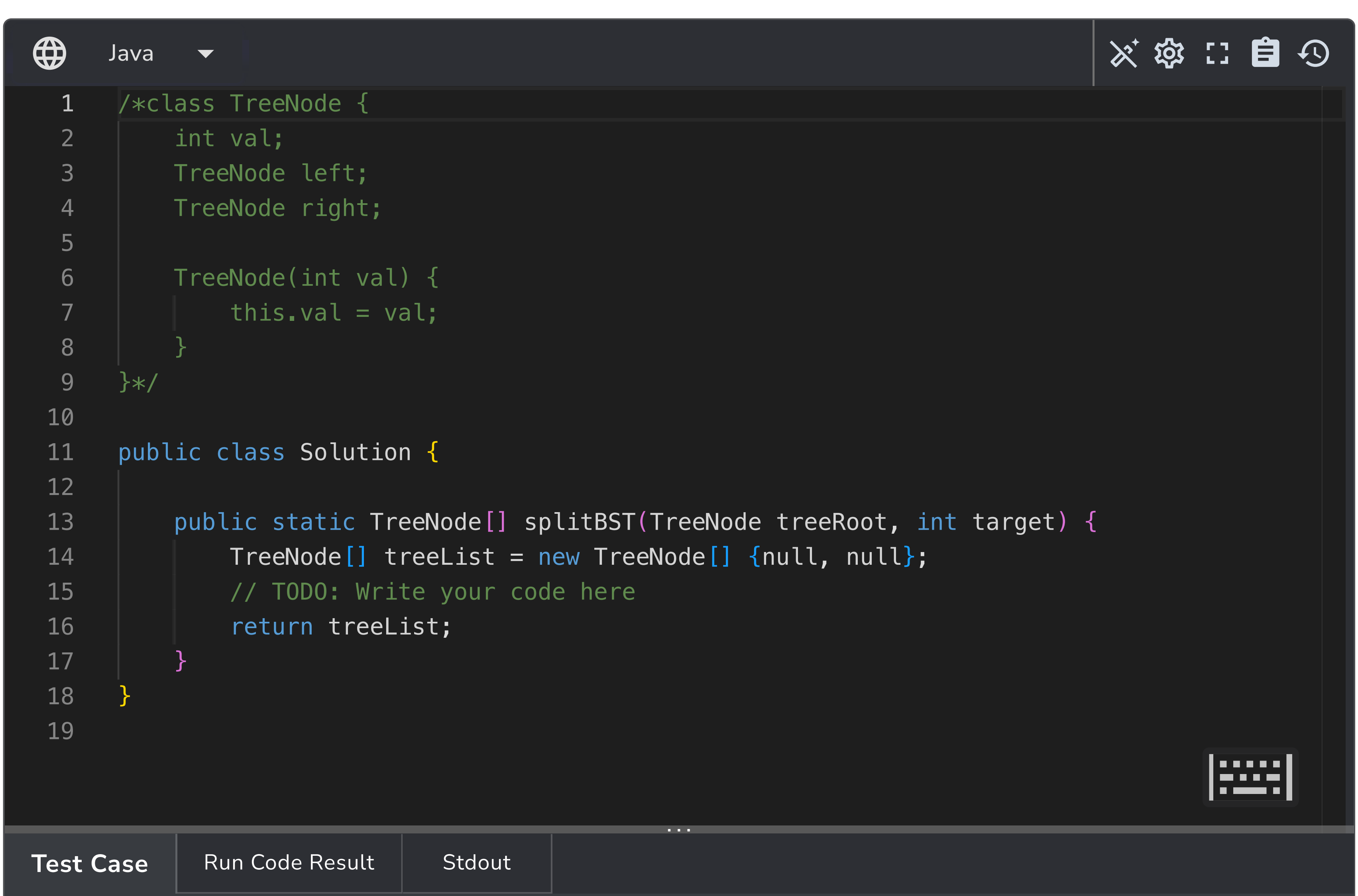
      ```
          4
         / \
        2   6
       /   /
      1   3
      ```

- Explanation:
  - In this example, the binary search tree is split based on the target value of 7.
  - Only the root node (7) is greater than the target value and is moved to Tree 1.
  - The remaining nodes (4, 2, 6, 1, 3) are smaller than or equal to the target value and are moved to Tree 2.
  - The resulting Tree 1 contains only the root node 7, while Tree 2 contains all other nodes from the original tree.

The resulting subtrees are then connected to the appropriate sides of the root node, forming the split BST.

Try it yourself

Try solving this question here:



```java
/*class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    TreeNode(int val) {
        this.val = val;
    }
}*/

public class Solution {

    public static TreeNode[] splitBST(TreeNode treeRoot, int target) {
        TreeNode[] treeList = new TreeNode[] {null, null};
        // TODO: Write your code here
        return treeList;
    }
}
```

```
[4, 2, 6, 1, 3, 7]
5
```

← Previous
Solution: Good Number

Next →
Solution: Split BST

☐ Mark as Completed